

# The Trading Strategy — explained simply

---

## What the strategy does, in plain words

This explains the trading idea behind `Backtesting/engine.py` without the heavy maths. If you can picture a rubber band snapping back, you already understand the core of it.

---

## 1. The big idea in one minute

---

We trade **spreads** — the *gap* between two related oil prices (for example, July oil minus August oil). We don't bet on whether oil goes up or down. We only bet on whether that **gap goes back to normal** after it gets stretched.

Think of the gap as a rubber band. Most of the time it sits at a comfortable length. Now and then it gets **stretched too far** — too wide or too narrow. History says it usually snaps back. So when we see a stretched gap, we bet on the snap-back. That's the whole strategy:

**When the gap is unusually stretched, bet that it returns to normal.**

Everything else is just doing this *carefully* — measuring "stretched" properly, only acting when the odds are good, and getting out at the right time.

---

## 2. What we actually trade

---

Seven different gaps, all built from **WTI** (US oil) and **Brent** (global oil) futures:

- **Calendar gaps** — same oil, two different months (e.g. July vs August WTI).
- **Butterflies** — a three-month combination that checks if the *middle* month is out of line.
- **Brent vs WTI** — the gap between the two kinds of oil.

We never own oil outright. Each trade buys one contract and sells another at the same time, so the overall oil price cancels out and only the *gap* matters. That makes the bet smaller and steadier than betting on oil itself.

---

### 3. Step 1 — Work out what "normal" looks like

---

To know if a gap is stretched, we first need its **normal** level. We look at the gap's recent history (the last few hours of trading) and take the **middle value** — the typical level it's been hanging around.

One important detail: we use the **middle value (the median)**, not a plain average. Why? Because the average gets dragged around by the very spike we're trying to spot. The median quietly ignores one-off spikes, so our idea of "normal" stays honest. (In simple terms: one crazy day shouldn't change what we call normal.)

We also measure how much the gap normally wiggles. A gap that usually moves a lot needs to move *a lot* to count as "stretched"; a calm gap needs only a little. This keeps every gap on the same fair ruler.

---

### 4. Step 2 — Measure how stretched it is

---

We turn "how far from normal" into a single number called the **z-score**:

- A z-score near **0** = totally normal.
- **+1.5 or more** = unusually **high** (the gap is "rich" — too wide).
- **-1.5 or less** = unusually **low** (the gap is "cheap" — too narrow).

It just answers: *"how unusual is this, in terms of the gap's own normal wiggle?"*

---

### 5. Step 3 — Decide whether to trade

---

We open a trade only when **all** of these are true. If any one fails, we skip it.

**(a) The gap is genuinely stretched.** The z-score is at least 1.5 (either direction). This is the level that, in years of past data, reliably snapped back. If it's rich, we **sell** it; if it's cheap, we **buy** it.

**(b) The "big picture" model agrees.** Separately, a slower model looks at real-world causes — oil stockpiles, the strength of the dollar, market fear, the season — and also says whether the gap looks rich or cheap. We only trade when **both** the fast read and this slow read point the **same way**. If only the fast, twitchy signal says "stretched" but the big-picture model disagrees, we walk away. Two opinions that agree are much safer than one.

**(c) The move is worth the cost (when we count costs).** Trading isn't free — there's a small cost every time you buy and sell. So we only take a trade if the expected snap-back is at least **twice**

what it costs to make the trade. No point chasing a tiny move that the trading cost would eat. (This check is on only when we run the "with real costs" version — see §9.)

---

## 6. Step 4 — Get out at the right time

---

Once we're in a trade, we leave for one of four reasons:

1. **It worked (take profit).** The gap snapped back to normal. We take the win. Importantly, we wait for the **full** snap-back, not just a tiny bit of it — we want the whole move.
2. **We were wrong (stop loss).** Instead of snapping back, the gap stretched even further against us. At a set point we admit we're wrong and cut the trade. This stop is kept **tighter than the profit target**, so a typical win is bigger than a typical loss.
3. **It's taking too long (time stop).** Each gap has a typical "snap-back speed." If a trade drags on well past that without working, we stop waiting and get out. This avoids sitting in dead trades that slowly bleed.
4. **The day ends (flatten).** We never hold through the overnight or weekend gap — too much can happen while the market is closed. We close out before the break.

The first of these to happen ends the trade.

---

## 7. Why the "get out" rules matter so much

---

The first version of this strategy grabbed a **tiny** profit but allowed a **big** loss. That gave a flashy "we win most of the time" number — but each win was small and each loss was large, which is fragile.

The new rules flip that: we hold for the **full** snap-back (bigger wins) and cut losers **quickly and tightly** (smaller, faster losses). We win a bit *less often*, but we **make more when right and lose less when wrong**. That's a healthier, more durable edge — and the honest way to measure a strategy is the money it keeps, not how often it wins.

---

## 8. How big are the trades, and how do we keep score?

---

- We trade **one unit** of each gap at a time — small and steady.
- One contract covers 1,000 barrels, so a **\$1 move** in a gap is worth **\$1,000**.
- We start with a pretend **\$250,000** and track the running profit/loss.

- For every trade we record: which gap, buy or sell, in/out prices and times, why we exited, how long we held, the profit or loss, and the worst and best point it passed through.

The headline scores we care about:

- **Profit factor** — dollars won for every dollar lost (above 1 = making money).
- **Average profit per trade** — the real bottom line.
- **Biggest dip (drawdown)** — the worst losing stretch, i.e. how much pain it caused.

---

## 9. The most important honesty check: costs

By default we run the strategy **before trading costs** — this shows whether the *prediction* is any good. But we can also run it **with realistic costs** flipped on.

And here's the key finding: **with realistic costs, trading too often actually loses money**. The little snap-backs aren't big enough to cover the cost of all that buying and selling.

The fix isn't to give up — it's to **trade less, but only the best setups**. When we add the "big-picture model must agree" filter, the strategy drops from ~190 trades down to a few dozen high-quality ones — and *those* stay profitable even after costs, with a much smaller dip.

So the real lesson is simple: **the signal is real, but it's fragile. You make money by being picky, not by trading a lot.**

---

## 10. What we changed to make it better (before → after)

| What                    | Old version                               | New version                                                | Why it's better                                  |
|-------------------------|-------------------------------------------|------------------------------------------------------------|--------------------------------------------------|
| <b>Getting out</b>      | grab a sliver of profit, allow a big loss | wait for the full snap-back, cut losses tighter and sooner | bigger wins, smaller losses, fewer dead trades   |
| <b>Counting costs</b>   | ignored costs completely                  | can run with real costs + skip trades that aren't worth it | shows the truth: over-trading loses money        |
| <b>Finding "normal"</b> | plain average (easily fooled by spikes)   | the middle value + a "big-picture must agree" filter       | a cleaner normal, and only well-supported trades |

In numbers, on the test data: **before costs** the strategy looks great. **After costs**, the trade-everything version turns *negative*. Add the agreement filter and it goes back to **positive with a tiny dip** — by simply trading the best few setups instead of all of them.

---

## 11. The honest limits

---

1. **Not much data yet.** Only a few days of trading. It's a working method, not a final verdict — feed it more days to really trust the numbers.
2. **Small trade size.** We trade one unit, so the dollar profits look small. The point here is *does the idea work*, not *how rich does it make you*. Sizing up is a separate step.
3. **Oil only.** The data is just WTI and Brent crude — no diesel, petrol, or other products.
4. **Costs are an estimate.** The cost we use is a sensible guess; a real trading desk would plug in its actual costs.

Saying all this plainly is the point — a believable result is one that's honest about what it can and can't claim.

---

## 12. How to run it

---

```
python Backtesting/engine.py # before costs (the pure signal)
python Backtesting/engine.py --slip 0.01 # with real costs + only-worth-it trades
python Backtesting/engine.py --slip 0.01 --fund-gate # with costs + big-picture must agree
python Backtesting/engine.py --live # keep re-running on the latest data
```

The results land in `out/trades_log.md` (every trade in words), `out/trades.csv` (a spreadsheet), and feed the dashboard. The plain-English project overview is in `../PROJECT_REPORT.md`.